Linked Lists in Data Structures

Concepts, Types, Operations, and Python Practices

What is a Linked List?

- A linear data structure of connected nodes.
- Each node holds data and a reference (pointer) to the next.
- Unlike arrays, memory is not contiguous.

Comparison with Arrays

- Arrays: Contiguous memory, fixed size, fast random access.
- Linked Lists: Dynamic size, scattered memory, easier insertion/deletion.

Node Structure in Python

- class Node:
- def __init__(self, data):
- self.data = data
- self.next = None
- Each node has:
- data: Value
- next: Pointer to next node

Types of Linked Lists

- Singly Linked List
- Doubly Linked List
- Circular Linked List

Singly Linked List

- One pointer per node (to the next node)
- Last node points to None
- Python: head -> node1 -> node2 -> None

Doubly Linked List

- Each node points both ways: prev and next
- Allows backward and forward traversal

Circular Linked List

- Last node links back to the first node
- Can be singly or doubly circular
- Useful in cyclic buffers, task scheduling

Traversing a Linked List

- temp = head
- while temp:
- print(temp.data)
- temp = temp.next
- Start from the head
- Move node by node using next

Python Code - Insert at End

```
def append(head, data):
    new_node = Node(data)
    if not head:
        return new_node
    temp = head
    while temp.next:
        temp = temp.next
    temp.next = new_node
    return head
```

Deletion in Linked List

Adjust pointers to skip the deleted node

Edge cases: head, tail, single element

Python Code - Delete Node

```
def delete_node(head, key):
    if not head:
        return None
    if head.data == key:
        return head.next
    temp = head
    while temp.next and temp.next.data != key:
        temp = temp.next
    if temp.next:
        temp.next = temp.next.next
    return head
```

Searching in Linked List

Traverse and compare each node's data

Linear time complexity: O(n)

pythonCopyEditdef search(head, key):

```
    temp = head
    while temp:
        if temp.data == key:
            return True
        temp = temp.next
    return False
```

Advantages of Linked Lists

- Dynamic memory allocation
- Efficient insertions/deletions
- No need to shift elements

Disadvantages of Linked Lists

- More memory (due to pointers)
- No direct indexing (slower access)
- More complex implementation

Summary & Practice Ideas

- Know how to: traverse, insert, delete, search
- Practice:
- Build a list with input()
- Write reverse function
- Convert array to linked list